

Guia Completo: Integração do Llama/Ollama no AgroVision AI

Material de apoio para implementação prática do chat local com LLM, visão computacional e dashboard no projeto AgroVision AI.

Tecnologias centrais	Backend	Objetivo
YOLO • Ollama • Llama • SQLite	FastAPI + HTML/CSS/JS	Monitoramento visual + chat local

Resultado esperado: ao final deste guia, o sistema deverá exibir câmera ao vivo, registrar eventos no banco e responder perguntas em linguagem natural com base no último evento detectado.

1. Visão geral da arquitetura final

Atualmente, o fluxo principal do sistema funciona da seguinte forma:

- A câmera captura frames continuamente.
- O backend processa parte desses frames com YOLO.
- Quando encontra pessoa ou veículo, salva uma captura e registra um evento no SQLite.
- O frontend carrega o feed ao vivo da câmera, cards de eventos recentes, painel de status e chat com IA local.
- Quando o usuário faz uma pergunta no chat, o frontend envia a pergunta ao FastAPI, que busca o último evento, monta o contexto, consulta o Ollama e devolve a resposta ao navegador.

Frontend (HTML + CSS + JS)

|

v

FastAPI (app.py)

|

\

|

\-> SQLite (eventos)

|

+-> YOLO (detecção visual)

|

+-> Ollama -> Llama 3

2. Como era o projeto no início

No início, o projeto era mais próximo de um protótipo de monitoramento, com backend FastAPI, processamento de câmera com YOLO, persistência simples em SQLite, página HTML básica, tabela de eventos e um chat ainda sem integração robusta com Llama/Ollama.

Na versão inicial, a lógica de chat utilizava uma abordagem mais simples, similar ao exemplo abaixo:

```
from services.chat_service import ask_about_last_event
```

```
@app.post("/chat", response_model=ChatResponse)
```

```
def chat(req: ChatRequest):
```

```
    answer = ask_about_last_event(req.message)
```

```
    return ChatResponse(answer=answer)
```

3. Objetivo da evolução

1. Integrar IA local real com Llama via Ollama.
2. Melhorar a experiência da câmera para ficar mais próxima de uma transmissão ao vivo.

3. Melhorar a experiência do chat, inclusive no tempo de resposta percebido.
4. Melhorar o design para um visual mais tecnológico e mais apresentável.

4. Passo a passo para implementar o Llama no projeto

4.1. Preparar o ambiente Python

Abra o PowerShell dentro da pasta do projeto e prepare o ambiente virtual.

```
cd C:\projetos\agrovision_ia
python -m venv .venv
.\venv\Scripts\Activate.ps1
pip install -r requirements.txt
```

4.2. Instalar e subir o Ollama

Depois de instalar o Ollama no Windows, inicie o serviço e baixe o modelo desejado.

```
ollama serve

ollama pull llama3
ollama list
```

4.3. Testar a API do Ollama antes da integração

Antes de ligar o backend ao modelo, valide se a API local está respondendo.

```
Invoke-WebRequest -UseBasicParsing http://127.0.0.1:11434/api/tags

$body = @{
    model='llama3'
    messages=@(@{role='user';content='Responda apenas OK'})
    stream=$false
} | ConvertTo-Json -Depth 5

Invoke-RestMethod -Method Post `
    -Uri http://127.0.0.1:11434/api/chat `
    -ContentType 'application/json' `
    -Body $body
```

4.4. Configurar o backend para usar o Ollama

As variáveis abaixo permitem trocar URL, modelo e tempo limite sem editar diretamente a lógica principal.

```
OLLAMA_URL = os.getenv("OLLAMA_URL", "http://127.0.0.1:11434/api/chat")
MODEL_NAME = os.getenv("OLLAMA_MODEL", "llama3")
OLLAMA_TIMEOUT = int(os.getenv("OLLAMA_TIMEOUT", "120"))
```

```
OLLAMA_KEEP_ALIVE = os.getenv("OLLAMA_KEEP_ALIVE", "30m")
```

4.5. Criar os modelos de request/response do chat

Esses modelos organizam a pergunta, o histórico e a resposta da IA.

```
class Message(BaseModel):
    role: str
    content: str

class ChatRequest(BaseModel):
    message: str
    history: List[Message] = []

class ChatResponse(BaseModel):
    answer: str
    history: List[Message]
```

4.6. Buscar o contexto do último evento no banco

Antes de chamar o modelo, o backend consulta o último evento salvo no banco para enriquecer o contexto.

```
def get_last_event():
    conn = sqlite3.connect(DB_PATH)
    conn.row_factory = sqlite3.Row
    cur = conn.cursor()

    cur.execute("""
        SELECT id, event_time, label, confidence, image_path
        FROM events
        ORDER BY id DESC
        LIMIT 1
    """)

    row = cur.fetchone()
    conn.close()

    if row:
        return dict(row)
    return None
```

4.7. Montar as mensagens enviadas para o Llama

A função abaixo define a instrução do sistema, injeta o último evento detectado e adiciona a nova pergunta do usuário.

```
def build_chat_messages(message: str, history: List[Message]):
```

```

system_message = {
    "role": "system",
    "content": "Você é um assistente do sistema AgroVision. Responda em português, de forma clara, objetiva e útil."
}

last_event = get_last_event()
if last_event:
    event_ctx = (
        f"Último evento detectado:\n"
        f"- ID: {last_event['id']}\n"
        f"- Horário: {last_event['event_time']}\n"
        f"- Objeto: {last_event['label']}\n"
        f"- Confiança: {last_event['confidence']:.2f}\n"
        f"- Imagem: {last_event['image_path']}\n"
    )
    history = [Message(role="system", content=event_ctx)] + history

return [system_message] + [h.dict() for h in history] + [{"role": "user", "content": message}]

```

4.8. Enviar a pergunta ao Ollama no backend

Nesta etapa, o sistema realiza a chamada HTTP ao Ollama e devolve a resposta pronta.

```

def ask_ollama(message: str, history: List[Message]):
    messages = build_chat_messages(message, history)
    payload = {
        "model": MODEL_NAME,
        "messages": messages,
        "stream": False,
        "keep_alive": OLLAMA_KEEP_ALIVE
    }

    with chat_request_scope():
        response = requests.post(OLLAMA_URL, json=payload, timeout=(10, OLLAMA_TIMEOUT))
        response.raise_for_status()
        data = response.json()
        return data.get("message", {}).get("content", ""), messages

```

4.9. Criar o endpoint REST normal do chat

Esse endpoint funciona como fallback e devolve a resposta completa de uma vez.

```

@app.post("/chat", response_model=ChatResponse)
def chat(req: ChatRequest):
    try:

```

```

    answer, messages = ask_ollama(req.message, req.history)
    new_history = req.history + [Message(role="assistant", content=answer)]
    return ChatResponse(answer=answer, history=new_history)
except Exception as e:
    return JSONResponse(status_code=500, content={"error": str(e)})

```

4.10. Criar o endpoint de streaming do chat

Para melhorar a experiência do usuário, a resposta pode ser enviada em partes.

```

@app.post("/chat/stream")
def chat_stream(req: ChatRequest):
    return StreamingResponse(
        stream_ollama(req.message, req.history),
        media_type="application/x-ndjson"
    )

def stream_ollama(message: str, history: List[Message]):
    messages = build_chat_messages(message, history)
    payload = {
        "model": MODEL_NAME,
        "messages": messages,
        "stream": True,
        "keep_alive": OLLAMA_KEEP_ALIVE
    }

```

4.11. Verificar a saúde do Ollama no backend

A rota abaixo ajuda a validar se o modelo local está acessível.

```

def get_ollama_status():
    base_url = OLLAMA_URL.rsplit("/api/", 1)[0]
    tags_url = f"{base_url}/api/tags"

@app.get("/health")
def health():
    return {"status": "ok", "ollama": get_ollama_status()}

```

4.12. Aquecer o modelo para reduzir a lentidão da primeira resposta

O warmup faz uma chamada inicial para diminuir o custo da primeira pergunta real.

```

def warmup_ollama():
    try:
        ask_ollama("Responda apenas: pronto", [])
        print(f'Ollama aquecido com o modelo {MODEL_NAME}.')
    except Exception as exc:
        print(f'Warmup do Ollama falhou: {exc}')

```

```
warmup_thread = threading.Thread(target=warmup_ollama, daemon=True)
warmup_thread.start()
```

5. Como o frontend foi implementado

5.1. Estrutura da tela atual

A interface foi reorganizada com hero de métricas, coluna principal para câmera e capturas, coluna lateral para assistente e cards com eventos recentes.

```
<link rel="stylesheet" href="/static/dashboard.css">
...
<div class="shell">
  <header class="hero panel">
```

5.2. Câmera ao vivo no frontend

Esse img aponta para o endpoint MJPEG e mantém o painel visual em atualização contínua.

```
<div class="video-shell">
  
</div>

@app.get("/video_feed")
def video_feed():
    return StreamingResponse(
        generate_mjpeg_stream(),
        media_type="multipart/x-mixed-replace; boundary=frame"
    )
```

5.3. Campo de pergunta do chat

A área do chat possui campo de entrada, botão de envio, status e caixa de resposta.

```
<textarea id="chat-question" placeholder="Exemplo: descreva o último evento e o nível de
confiança."></textarea>
<button id="chat-send-button" class="primary-btn" onclick="sendQuestion()">Enviar pergunta</button>
<div id="chat-status" class="chat-status">Pronto para consultar o modelo local.</div>
<div id="chat-response" class="chat-response">A resposta do sistema aparecerá aqui...</div>
```

5.4. JavaScript do chat

O JavaScript foi extraído para arquivo separado e passou a tratar streaming, fallback e mensagens de status.

```
async function readStreamedChat(response, responseBox, statusBox, startedAt) {
    const reader = response.body.getReader();
    const decoder = new TextDecoder();
```

```

}

async function fallbackChat(message, responseBox, statusBox, startedAt) {
  const response = await fetch("/chat", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ message: message })
  });
}

async function sendQuestion() {
  const response = await fetch("/chat/stream", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ message: message })
  });
}

```

5.5. CSS separado e visual mais tecnológico

O CSS passou a concentrar grid, gradientes, cards e a estética principal do dashboard.

```

:root {
  --bg-a: #eff6fb;
  --bg-b: #f8fbff;
  --panel: rgba(255, 255, 255, 0.82);
  --accent: #0284c7;
}

.hero {
  display: grid;
  grid-template-columns: minmax(0, 1.35fr) minmax(320px, 0.95fr);
}

.event-grid {
  display: grid;
  grid-template-columns: repeat(2, minmax(0, 1fr));
}

```

6. Linha do tempo da evolução do projeto

Fase	Resumo
Fase 1. Base de monitoramento	FastAPI, YOLO, SQLite, listagem simples de eventos e página HTML simples.

Fase 2. Integração inicial do Ollama	Definição de OLLAMA_URL, criação de modelos de chat, função ask_ollama e rota /chat.
Fase 3. Robustez do backend	Health check, warmup, mensagens de erro melhores e ajuste do run.ps1.
Fase 4. Câmera ao vivo de verdade	Uso de StreamingResponse, MJPEG e rota /video_feed.
Fase 5. UX do chat melhorada	Botão desabilitado durante consulta, status, tempo de resposta e streaming.
Fase 6. Frontend redesenhado	Novo index.html, novo CSS, novo JS e substituição de tabela por cards.
Fase 7. Performance entre YOLO e Ollama	Ajuste de intervalos de detecção e parâmetros para reduzir disputa por recursos.

7. Comandos principais para rodar o projeto

Ativar ambiente

```
cd C:\projetos\agrovision_ia  
.\venv\Scripts\Activate.ps1
```

Subir o Ollama

```
ollama serve
```

Garantir que o modelo existe

```
ollama pull llama3  
ollama list
```

Rodar o FastAPI

```
.\venv\Scripts\uvicorn.exe app:app --host 127.0.0.1 --port 8001 --reload
```

Abrir no navegador

```
http://127.0.0.1:8001
```

Verificar health

```
Invoke-RestMethod http://127.0.0.1:8001/health
```

8. Recorte mínimo do backend

```
OLLAMA_URL = "http://127.0.0.1:11434/api/chat"  
MODEL_NAME = "llama3"  
  
def build_chat_messages(message, history):
```

```

last_event = get_last_event()
system_message = {
    "role": "system",
    "content": "Você é um assistente do AgroVision."
}

messages = [system_message]

if last_event:
    messages.append({
        "role": "system",
        "content": f"Último evento: {last_event['label']} em {last_event['event_time']}"
    })

messages.extend([h.dict() for h in history])
messages.append({"role": "user", "content": message})
return messages

def ask_ollama(message, history):
    payload = {
        "model": MODEL_NAME,
        "messages": build_chat_messages(message, history),
        "stream": False
    }
    response = requests.post(OLLAMA_URL, json=payload)
    response.raise_for_status()
    return response.json()["message"]["content"]

```

9. Recorte mínimo do frontend

```

<textarea id="chat-question"></textarea>
<button onclick="sendQuestion()">Enviar</button>
<div id="chat-response"></div>
async function sendQuestion() {
    const message = document.getElementById("chat-question").value.trim();

    const response = await fetch("/chat", {
        method: "POST",
        headers: {
            "Content-Type": "application/json"
        },
        body: JSON.stringify({ message: message })
    })

```

```
});  
  
const data = await response.json();  
document.getElementById("chat-response").innerText = data.answer;  
}
```

10. Ponto importante sobre o estado atual do app.py

O arquivo app.py ainda possui funções duplicadas por causa da evolução incremental do projeto. Em Python, vale a última definição da função. Por isso, as funções realmente utilizadas costumam estar mais abaixo no arquivo.

- build_chat_messages
- ask_ollama
- process_stream

11. Resumo da transformação do projeto

- monitoramento com YOLO
- câmera ao vivo
- eventos persistidos em banco
- IA local com Llama via Ollama
- painel de status
- resposta por streaming
- dashboard moderno com aparência mais tecnológica

12. Próximas melhorias recomendadas

5. limpar duplicações do app.py
6. quebrar o projeto em módulos dentro de services/
7. permitir escolher o modelo do Ollama pelo frontend
8. salvar histórico completo de conversa
9. mostrar tempo de primeira palavra e tempo total do LLM
10. criar uma página de configurações

13. Passo a passo copiável para execução

Abrir a pasta do projeto

```
cd C:\projetos\agrovision_ia
```

Ativar o ambiente virtual

```
.\venv\Scripts\Activate.ps1
```

Se der erro de política de execução

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
```

```
.\venv\Scripts\Activate.ps1
```

Subir o Ollama

```
ollama serve
```

Garantir que o modelo existe

```
ollama pull llama3
```

```
ollama list
```

Rodar o projeto no modo recomendado

```
.\run.ps1 -Foreground
```

Abrir no navegador

```
http://127.0.0.1:8001
```

Encerrar ao final

```
Ctrl + C
```

14. O que mudou no script run.ps1

O script agora pode rodar em background ou foreground. O modo foreground é o mais indicado para desenvolvimento, pois permite encerrar o servidor com Ctrl + C.

```
if ($Foreground) {  
    Write-Host "Iniciando FastAPI (uvicorn) em foreground..."  
    Write-Host "Use Ctrl + C para encerrar o servidor."  
  
    try {  
        & $uvicornExe "app:app" "--host" "127.0.0.1" "--port" "8001" "--reload"  
    } finally {  
        if ($startedOllamaProcess -and -not $startedOllamaProcess.HasExited) {  
            Stop-Process -Id $startedOllamaProcess.Id -Force -ErrorAction SilentlyContinue  
        }  
    }  
}
```

15. Resultado final esperado

11. subir o Ollama

12. subir o FastAPI
13. abrir o dashboard
14. visualizar a câmera ao vivo
15. visualizar capturas recentes
16. visualizar eventos recentes em cards
17. realizar perguntas no chat
18. receber respostas do Llama local com contexto do último evento
19. perceber a resposta surgindo por streaming

16. Desafio final

Desafio: implementar histórico de conversa com contexto persistente.

Objetivo: fazer com que o sistema mantenha um histórico real das últimas interações do chat, em vez de responder apenas com base na pergunta atual e no último evento.

Requisitos mínimos:

- Salvar as mensagens do usuário e da IA em memória ou em banco.
- Enviar esse histórico aos endpoints /chat e /chat/stream.
- Mostrar no frontend as últimas mensagens em formato de conversa.
- Permitir que o usuário continue uma conversa anterior.
- Garantir que o sistema continue usando também o contexto do último evento detectado.

Nível extra:

- opção para limpar o histórico no frontend
- mostrar tempo de resposta total da IA
- mostrar tempo até a primeira palavra
- permitir escolher entre dois modelos no Ollama
- criar uma página de configurações do sistema

Entrega esperada: um sistema com câmera ao vivo, eventos registrados, chat local funcional e histórico visível na interface.